

PRD · v0.4 · 4 października 2026

Night Maze

Gra 3D w OpenGL 4.1 / C++: nocny labirynt, latarka, zbieranie kryształów. Projekt zaliczeniowy z przedmiotu OpenGL (UG), budowany od pierwszego tygodnia semestru.

Autor	Kacper „Shirone” (projekt indywidualny)
Platformy	macOS (MacBook, Apple Silicon) + Windows (główny PC), OpenGL 4.1 Core na obu
Język i build	C++20, CMake + FetchContent
Biblioteki	GLFW, GLAD, GLM, Dear ImGui, stb_image, opcjonalnie Assimp; własny loader OBJ
Assety	Blender (sterowany przez Blender MCP) → eksport OBJ/MTL
Cel	Projekt na ocenę z wykładu + baza kodu do zadań laboratoryjnych
Dokumentacja	Każdy moduł i każda biblioteka ma dokument do nauki w docs/ (sekcja 7)

Zasada przewodnia: każdy z 15 tematów wykładu staje się jedną widoczną funkcją gry, którą da się włączyć, wyłączyć i pokazać w panelu ImGui podczas obrony.

1. Wizja i koncept

Gracz budzi się w kamiennym labiryncie w środku nocy. Ma tylko latarkę z ograniczoną baterią. Aby otworzyć wyjście, musi znaleźć określoną liczbę świecących kryształów rozrzuconych po korytarzach. Kryształy są jednocześnie źródłami światła i miejscami ładowania baterii, więc ciemność jest głównym przeciwnikiem.

Klimat: chłodne światło księżyca, mgła przy ziemi, ciepła plama latarki z ostrymi cieniami, pulsujące kryształy z poświatą (bloom). Gra jest krótka (5–10 minut na rundę), labirynt generowany proceduralnie z seeda.

Cele projektu

- **Zaliczenie:** pokryć wszystkie tematy wykładu w jednej spójnej aplikacji, którą łatwo obronić.
- **Nauka:** zrozumieć programowalny potok OpenGL od zera, bez silnika.
- **Reużycie:** warstwa *engine* służy też do zadań domowych z laboratorium.
- **Jakość kodu:** czysta, warstwowa struktura, którą można pokazać w portfolio.
- **Dokumentacja do nauki:** z dokumentów w docs/ da się nauczyć OpenGL, ImGui i reszty stosu oraz przygotować do kartkówki i obrony.

Poza zakresem (non-goals)

- Fizyka ciał sztywnych, animacja szkieletowa, sieć, dźwięk przestrzenny (dźwięk co najwyżej prosty, opcjonalnie).
- Edytor poziomów; Linux jako wymaganie (CMake powinien działać, ale testujemy na macOS i Windows).
- Vulkan/Metal, PBR – zostajemy przy modelu Phong'a, bo tego dotyczy wykład.

2. Gameplay

Pętla rozgrywki

Eksplorej korytarz → zauważ poświatę kryształu → zbierz go (bateria +, licznik +) → po zebraniu N kryształów wyjście się otwiera → dotrzyj do wyjścia, zanim bateria się skończy.

Mechanika	Opis	Priorytet
Ruch FPS	WASD + mysz, sprint, kolizja ze ścianami (ślizganie się wzdłuż ściany)	MUST
Latarka	Spotlight przypięty do kamery, klawisz F, bateria spada w czasie, migotanie przy niskim stanie	MUST
Kryształy	Point light + poświata, animacja unoszenia i obrotu, zbieranie po kolizji sferycznej	MUST
Wyjście	Brama otwiera się po zebraniu N kryształów; wejście w strefę kończy rundę	MUST
Generator labiryntu	Recursive backtracker na siatce, seed w ImGui, przycisk „Regeneruj”	MUST
Interakcja	Klik myszą na dźwignie/notatki na ścianach (selekcja obiektów)	SHOULD
Minimapa	Odkryte korytarze renderowane z góry do osobnego FBO	SHOULD
Przeciwnik	Prosty „cień” podążający BFS-em po siatce, gaśnie w świetle latarki	COULD
Ekran	Menu start / koniec rundy w ImGui (czas, zebrane kryształy)	COULD

3. Mapowanie na tematy wykładu

Każdy temat ma swoją funkcję w grze i przełącznik w panelu debug, żeby na obronie pokazać efekt „przed i po”.

#	Temat wykładu	Realizacja w Night Maze	Pokaz w ImGui
1	Pierwszy program OpenGL	Okno GLFW, kontekst 4.1 Core, pętla gry z fixed timestep	FPS, czas klatki
2	Programowalny potok	Klasa Shader, hot-reload GLSL z dysku	Przycisk „Reload shaders”
3	Przekształcenia przestrzeni	Model/View/Projection, kamera FPS, hierarchia transformów	Pozycja/rotacja kamery, FOV
4	Wczytywanie OBJ	Własny parser OBJ/MTL (v, vt, vn, f, usemtl), cache meshy	Lista załadowanych modeli
5	Tekstury	stb_image, mipmapy, filtrowanie anizotropowe, normal mapy ścian	Podgląd tekstur, toggle normal map
6	Światło kierunkowe i punktowe	Księżyc (kierunkowe), kryształy i pochodnie (punktowe), latarka (spot)	Edytor światel: kolor, siła, tłumienie
7	Gouraud vs Phong	Dwa warianty shadera oświetlenia, przełączane w locie	Combo: Gouraud / Phong / Blinn-Phong
8	Tekstura sześcienna	Skybox z nocnym niebem i gwiazdami	Toggle skybox
9	Shader geometrii	Kępki trawy przy ścianach generowane z punktów; iskry wokół kryształów	Gęstość trawy, toggle
10	Rendering pozaekranowy	Scena do HDR FBO → post-process (bloom, mgła, winieta); minimapa	Podgląd załączników FBO
11	Shadow mapping	Cienie z latarki (perspektywiczne) i księżyc (ortograficzne), PCF	Podgląd shadow mapy, bias
12	Environment mapping	Kryształy i kałuże odbijające/załamujące skybox	Współczynnik odbicia/refrakcji
13	Implementacja podłoża	Teren z heightmapy pod labiryntem, wysokość gracza z terenu	Skala wysokości, wireframe
14	Wstęp do kolizji	AABB gracz vs ściany (siatka), sfery dla kryształów	Debug draw brył kolizji
15	Selekcja obiektów	Ray casting z kamery (lub color picking) do dźwigni i notatek	Podświetlenie wybranego obiektu

4. Platformy: macOS + Windows, OpenGL 4.1

Najważniejsze ograniczenie: macOS obsługuje maksymalnie **OpenGL 4.1 Core**, więc to wspólny mianownik także na Windowsie (który ma 4.6). Wszystkie tematy wykładu się w tym mieszczą (shadery geometrii są od 3.2), ale część tutoriali używa funkcji z 4.3+, które na Macu nie zadziałają.

Temat	Co to oznacza w praktyce
Kontekst	glfwWindowHint: CONTEXT_VERSION 4.1, OPENGGL_CORE_PROFILE, OPENGGL_FORWARD_COMPAT = GL_TRUE
GLAD	Wygenerować loader dla GL 4.1 Core (glad2 lub webowy generator), bez rozszerzeń
GLSL	Wszystkie shadery: #version 410 core; ImGui backend z "#version 410"
Brak 4.3+	Nie ma glDebugMessageCallback, compute shaderów, SSBO, glObjectLabel. Zamiast tego makro GL_CHECK z glGetError w buildach Debug
Retina	Rozmiar framebuffera ≠ rozmiar okna. glViewport i FBO zawsze z glfwGetFramebufferSize, mysz w koordynatach okna
Deprecation	Zdefiniować GL_SILENCE_DEPRECATED, żeby wyciszyć ostrzeżenia Apple
Linie	glLineWidth > 1 nie działa w Core na Macu; debug draw robimy cienkimi liniami lub quadami
Narzędzia	RenderDoc nie wspiera macOS – na Macu debug przez ImGui i GL_CHECK; na Windowsie dodatkowo RenderDoc
Windows	Visual Studio 2022 (Desktop C++) lub CLion; ten sam CMakePresets.json; .gitattributes z *text=auto; ścieżki tylko przez std::filesystem
Workflow	Git od pierwszego dnia: praca na Macu i Windowsie przez push/pull, build sprawdzany na obu przed tagiem kamienia milowego

5. Stos technologiczny

Element	Wybór	Uzasadnienie
Język	C++20	Wykład w C/C++; std::span, designated initializers, concepts tam, gdzie pomagają
Build	CMake ≥ 3.24 + FetchContent	Jedno polecenie konfiguracji, zależności pobierane automatycznie, przypięte do tagów
Okno / input	GLFW 3.4	Standard z wykładów i LearnOpenGL
Loader GL	GLAD (4.1 Core)	Wymóg z zajęć
Matematyka	GLM	Zgodne z GLSL typy i funkcje
Debug UI	Dear ImGui (docking branch)	Panele debug, edytor świateł, podgląd FBO
Obrazy	stb_image	Jeden nagłówek, PNG/JPG/HDR
Modele	Własny loader OBJ + opcjonalnie Assimp	Loader OBJ to temat wykładu nr 4, więc piszemy go sami; Assimp dozwolony np. dla bardziej złożonych modeli
Jakość	clang-format, clang-tidy, -Wall -Wextra -Wpedantic	Spójny styl, wczesne łapanie błędów
IDE	CLion / VS Code + clangd (Mac), Visual Studio 2022 (Windows)	CMake presets dla Debug/Release na obu systemach

Element	Wybór	Uzasadnienie
Dokumentacja	Markdown w docs/ + komentarze Doxygen w nagłówkach	Czytelne na GitHubie i w IDE, Mermaid do diagramów

6. Architektura

Kod dzielimy na warstwy z jednokierunkowymi zależnościami. Niższa warstwa nigdy nie wie o wyższej, dzięki czemu **engine/** można bez zmian użyć w zadaniach laboratoryjnych.

game/	logika Night Maze: labirynt, gracz, kryształy, latarka, stany gry
scene/	encje, transformy, kamera, światła, kolizje, selekcja
renderer/	passy renderowania, materiały, shadow mapy, post-process
gfx/	cienkie wrappery RAII na obiekty GL: Shader, Buffer, VertexArray, Texture2D, Cubemap, Framebuffer
core/	okno, input, czas, logowanie, ścieżki do assetów, GL_CHECK
debug/	panele ImGui (zależy od wszystkich, nic nie zależy od niego)

Zasady

- **RAII dla obiektów GL:** każda klasa z gfx/ tworzy obiekt w konstruktorze, usuwa w destruktorze, jest move-only.
- **Brak globali:** jeden obiekt *Application* posiada okno, renderer i stan gry; zależności przekazywane jawnie.
- **Dane zamiast kodu:** parametry świateł, materiałów i labiryntu w strukturach edytowalnych z ImGui (opcjonalnie JSON).
- **Shadery jako pliki:** GLSL w *assets/shaders*, wspólne fragmenty przez prosty `#include` w loaderze.
- **Jeden plik = jedna odpowiedzialność:** nagłówki w *include/* nie są potrzebne – `.hpp/.cpp` obok siebie w *src/*.

Struktura katalogów

```
night-maze/
├── CMakeLists.txt
├── CMakePresets.json          # debug / release
├── .clang-format .clang-tidy .gitignore
├── cmake/
│   └── Dependencies.cmake    # FetchContent: glfw, glm, imgui, stb
├── external/
│   └── glad/                 # wygenerowany GLAD 4.1 Core
├── assets/
│   ├── models/              # *.obj + *.mtl z Blendera
│   ├── textures/            # albedo, normal, heightmap
│   ├── skybox/              # px nx py ny pz nz
│   └── shaders/
│       ├── common/          # lighting.glsl, shadows.glsl, fog.glsl
│       ├── lit.vert/.frag    # Phong / Blinn-Phong
│       ├── gouraud.vert/.frag
│       ├── shadow_depth.vert/.frag
│       ├── skybox.vert/.frag
│       ├── grass.vert/.geom/.frag
│       ├── terrain.vert/.frag
│       ├── reflect.vert/.frag
│       └── post/ bloom, composite, minimap
├── src/
│   ├── main.cpp
│   ├── core/                Application, Window, Input, Time, Log, GlCheck
│   ├── gfx/                 Shader, Buffer, VertexArray, Texture2D, Cubemap, Framebuffer
│   ├── assets/              ObjLoader, ImageLoader, AssetCache
│   ├── renderer/            Renderer, ShadowPass, ScenePass, SkyboxPass,
│   │                       GrassPass, PostProcess, Material, Mesh
│   ├── scene/               Transform, Camera, Light, Collider, Raycast, Scene
│   ├── game/                Maze, MazeGenerator, Player, Flashlight, Crystal,
│   │                       Exit, GameState
│   └── debug/               DebugUI, panele: Renderer, Lights, Camera, Maze, Fbo
├── tools/
│   └── blender/              # skrypty eksportu i notatki do Blender MCP
├── tests/                   # testy loadera OBJ i generatora labiryntu
└── docs/
    ├── PRD.pdf
    ├── README.md            # spis treści i ścieżka nauki
    ├── syllabus.md          # temat wykładu → dokument → pliki kodu
    ├── modules/              # core, gfx, assets, renderer, scene,
    │                       # game, debug + każdy pass renderowania
    ├── libraries/            # glfw, glad, glm, imgui, stb_image, assimp
    ├── decisions/            # krótkie notatki: dlaczego tak, a nie inaczej
    └── guides/               # build-macos, build-windows, blender,
                            # debugging, glossary (słowniczek PL/EN)
```

7. Dokumentacja jako materiał do nauki

Definition of Done: moduł jest skończony dopiero wtedy, gdy ma dokument w docs/, aktualizowany w tym samym commicie co kod. Dokumenty pisane są po polsku i tak, żeby z samej ich lektury dało się nauczyć danego zagadnienia i przygotować do kartkówki.

Szablon dokumentu modułu (docs/modules/<moduł>.md)

Sekcja	Co zawiera
1. Po co to jest	Jeden akapit: jaki problem rozwiązuje moduł i który temat wykładu realizuje
2. Teoria	Pojęcia i matematyka (macierze, wektory, równania oświetlenia) z diagramem
3. Jak to działa w OpenGL	Wywołania gl* w kolejności, jaki stan GL jest ustawiany, co tworzymy, wiążemy i zwalniamy
4. Shadery	Wejścia, wyjścia, uniformy; kluczowe fragmenty GLSL objaśnione linijka po linijce
5. Kod w projekcie	Klasy i pliki, przepływ danych, gdzie moduł jest wywoływany; odnośniki do kodu
6. Panel ImGui	Co można zmieniać w locie i czego uczy obserwacja efektu
7. Pułapki	Typowe błędy (czarny ekran, złe normalne, shadow acne), różnice macOS/Windows
8. Ćwiczenia	2-4 małe modyfikacje do samodzielnego wykonania, żeby sprawdzić zrozumienie
9. Pytania kontrolne	Pytania w stylu kartkówki/obrony z krótkimi odpowiedziami
10. Źródła	Rozdział LearnOpenGL, docs.gl, książki z sylabusu (Ganczarski, Księga eksperta)

Dokumenty bibliotek (docs/libraries/)

GLFW, GLAD, GLM, Dear ImGui, stb_image i Assimp (jeśli użyty) dostają własny dokument: czym jest biblioteka, jak ją podpinamy w CMake, najważniejsze API z przykładami z naszego kodu, pułapki i linki do oficjalnej dokumentacji. Dokument ImGui opisuje dodatkowo model immediate mode, pętlę NewFrame/Render, docking i jak dodać nowy panel.

Pozostała dokumentacja

- **docs/README.md** – spis treści i kolejność czytania zgodna z kolejnością wykładów.
- **docs/syllabus.md** – tabela: temat wykładu → dokument → pliki kodu → przełącznik w ImGui. Ściąga przed kartkówką i obroną.
- **docs/guides/** – budowanie na macOS i Windowsie, pipeline Blender → OBJ, debugowanie GL, słowniczek PL/EN (VAO, VBO, FBO, UBO...).
- **Komentarze w kodzie** – każdy plik zaczyna się krótkim opisem i odnośnikiem do dokumentu; publiczne API z komentarzami Doxygen.
- **docs/decisions/** – krótkie notatki, dlaczego wybraliśmy dane rozwiązanie (np. AABB zamiast silnika fizyki).

Lista dokumentów modułów

Moduł	Dokumenty	Tematy
core	core.md – okno, kontekst GL, pętla gry, input, czas, GL_CHECK	1
gfx	shaders.md, buffers-vao.md, textures.md, framebuffers.md	2, 5, 8, 10
assets	obj-loader.md, images.md	4, 5
scene	transforms-camera.md, lights.md, collision.md, picking.md	3, 6, 14, 15

Moduł	Dokumenty	Tematy
renderer	lighting-gouraud-phong.md, shadows.md, skybox.md, grass-geometry.md, env-mapping.md, post-process.md, terrain.md	6-13
game	maze-generator.md, player.md, flashlight.md, gameplay.md	-
debug	debug-ui.md – architektura paneli ImGui, jak dodać nowy panel	-

8. Potok renderowania (klatka)

Pass	Wejście → wyjście	Tematy
1. Shadow pass	Geometria → depth texture latarki (perspektywa) i księżyc (orto)	11
2. Scene pass	Teren, ściany, modele OBJ z oświetleniem Phong/Gouraud + cienie → HDR FBO (RGBA16F)	3-7, 11, 13
3. Trawa	Punkty → geometry shader → kępki quadów, ten sam HDR FBO	9
4. Odbicia	Kryształy i kałuże próbujące cubemapę (reflect/refract)	12
5. Skybox	Cubemap rysowana na końcu z depth = 1.0	8
6. Post-process	HDR FBO → bright pass → blur → bloom; mgła z głębi, winieta, tone mapping → ekran	10
7. Minimapa	Widok z góry do małego FBO, nakładany w rogu	10
8. ImGui	Panele debug na wierzchu	-

Model oświetlenia

Jeden uniform buffer z tablicą światła (1 kierunkowe, do 16 punktowych, 1 spot). Kryształy mają tłumienie dobrane tak, żeby oświetlały ok. 1,5 komórki labiryntu. Latarka: stożek wewnętrzny/zewnętrzny z miękkim brzegiem i opcjonalną teksturą „cookie”. Mgła eksponencjalna liczona w post-processie z bufora głębi.

9. Pipeline assetów: Blender MCP → OBJ

Modele tworzymy w Blenderze, sterując nim przez Blender MCP z poziomu Claude, a potem eksportujemy do OBJ. Stylistyka low-poly z prostymi teksturami, żeby było szybko i spójnie.

Konwencja	Ustalenie
Osie	Y-up, -Z forward (opcje eksportu OBJ w Blenderze)
Skala	1 jednostka = 1 metr; komórka labiryntu = 2 × 2 m, ściana 3 m wysokości
Eksport	Triangulate faces, normals, UVs, materials (.mtl), Apply transforms, bez modyfikatorów
Origin	Na środku podstawy obiektu, żeby stawiać modele bezpośrednio na podłodze
Nazwy	snake_case: wall_straight.obj, crystal_01.obj, torch.obj, gate.obj, lever.obj
Tekstury	PNG, potęgi dwójki (512/1024), albedo + normal map, ścieżki względne w .mtl

Lista modeli (MVP)

Moduły ścian (prosta, róg, koniec), podłoga, kryształ (2 warianty), pochodnia, brama wyjścia, dzwignia, notatka, kamienie i chwasty jako dekoracja. Na start wszystko można zastąpić kostkami, a modele wymieniać stopniowo.

10. Panele debug (ImGui)

Panel	Zawartość
Renderer	FPS, czas klatki, draw calle, toggle każdego passu, tryb Gouraud/Phong/Blinn-Phong, wireframe
Lights	Lista świateł, kolor, intensywność, tłumienie, kąty latarki, kierunek księżyca
Shadows	Rozdzielczość, bias, PCF on/off, podgląd depth texture
Framebuffers	Podgląd HDR, bright pass, bloom, minimapy; exposure i próg bloomu
Camera	Pozycja, FOV, czułość myszy, tryb noclip (latanie przez ściany)
Maze	Seed, rozmiar, liczba kryształów, przycisk Regeneruj, widok siatki z góry
Collision	Debug draw AABB i sfer, wynik ostatniego raycasta
Shaders	Lista programów, przycisk Reload, ostatnie błędy kompilacji

Klawisz **F1** pokazuje/ukrywa całość; panele dokowane, żeby na obronie łatwo przełączać widoki.

11. Kamienie milowe

Harmonogram: oddanie najpóźniej w grudniu/styczniu, ale celem jest skończyć w **6-8 tygodni** (do końca listopada). Grywalne MVP po 3 tygodniach, wszystkie 15 tematów po 6 tygodniach, potem szlif i przygotowanie do code review. Grudzień i styczeń zostają jako bufor.

Projekt wyprzedza wykłady, więc część tematów poznajemy samodzielnie, zanim pojawią się na zajęciach. Dlatego dokumentacja modułu powstaje razem z kodem: to z niej uczymy się danego tematu. Każdy kamień kończy się działającą wersją na macOS i Windowsie, dokumentami modułów z tego etapu i commitem z tagiem.

Termin	Etap	Zakres	Kryterium ukończenia
4-5 X	M0	Repo, CMake + FetchContent, okno GLFW 4.1, GLAD, ImGui, GL_CHECK	Build na Macu (nd.) i Windowsie (pon.), panel z FPS; docs: core, glfw, glad, imgui, build
6-11 X	M1	gfx/: Shader, Buffer, VAO; trójkąt → kostka z MVP; kamera FPS	Latanie kamerą wokół kostki; docs: shaders, buffers-vao, transforms-camera, glm
12-18 X	M2 + M3	Generator labiryntu, kolizje AABB; tekstury, loader OBJ, pierwsze modele z Blendera	Chodzenie po teksturowanym labiryncie bez przenikania ścian
19-25 X	M4 + M5	Księżyc, latarka, kryształy, Gouraud vs Phong; zbieranie, bateria, brama	Grywalne MVP: pełna runda od startu do wyjścia
26 X-1 XI	M6	Skybox, teren z heightmapy, trawa w geometry shaderze	Przełączane w ImGui, udokumentowane
2-8 XI	M7	HDR FBO, bloom, mgła, minimapa; shadow mapping latarki i księżyca	Cienie i poświata w 60 FPS
9-15 XI	M8	Environment mapping, selekcja obiektów, dźwignie/notatki	Wszystkie 15 tematów odhaczone
16-29 XI	M9	Szlif, menu, balans, opcjonalny przeciwnik, README, przygotowanie do code review	Gotowe do oddania
XII-I	Bufor	Poprawki, dodatkowe wymagania prowadzącego, powtórka przed obroną	Obrona

Przygotowanie do obrony (code review + pytania)

Obrona polega na przeglądzie kodu i pytaniach prowadzącego, więc liczy się zrozumienie każdej linijki, nie tylko działająca gra.

- **Zasada zero tajemnic:** do repo nie trafia kod, którego nie potrafię wytłumaczyć. Kod pisany z pomocą AI czytam, przerabiam i opisuję w docs/, zanim go zcommituję.
- **Samodzielny code review przed każdym tagiem:** przejście diffu z checklistą (nazwy, RAIi, brak magicznych liczb, komentarze, zgodność z dokumentem).
- **Pytania kontrolne** z sekcji 9 każdego dokumentu modułu to gotowa lista do powtórki; w M9 odpowiadam na nie bez zaglądania w kod.
- **Mapa kodu:** docs/syllabus.md pozwala w kilka sekund otworzyć plik realizujący dany temat, gdy prowadzący o niego zapyta.
- **Historia commitów** małymi krokami, z opisowymi wiadomościami, pokazuje, że projekt powstawał stopniowo.

12. Wymagania niefunkcjonalne

- Stabilne **60 FPS** w 1440p (Retina) na MacBooku w buildzie Release.
- Build bez ostrzeżeń przy -Wall -Wextra; brak wycieków obiektów GL (RAII).
- Uruchomienie z dowolnego katalogu (ścieżka do assets/ ustalana względem pliku wykonywalnego).
- Błędy kompilacji shaderów wypisywane z nazwą pliku i numerem linii, bez crasha (hot-reload zachowuje poprzednią wersję).
- README z instrukcją budowania i listą sterowania; pełna dokumentacja modułów i bibliotek w docs/ według szablonu z sekcji 7.

13. Ryzyka i otwarte pytania

Ryzyko / pytanie	Plan
Biblioteki zewnętrzne	Rozstrzygnięte: dozwolone. Loader OBJ i tak piszemy sami, bo to temat wykładu
Dokumentacja odstaje od kodu	Dokument jest częścią Definition of Done; bez niego kamień milowy nie jest zamknięty
Różnice Mac/Windows	Build na obu systemach przed każdym tagiem; tylko OpenGL 4.1 i std::filesystem
Zakres przy pracy solo	Rozstrzygnięte: projekt indywidualny. Priorytet MUST przed efektami, kamienie milowe rozpisane pod jedną osobę
Termin i forma obrony	Rozstrzygnięte: oddanie XII/I, obrona = code review + pytania. Cel: skończyć do końca XI
Projekt wyprzedza wykłady	Uczymy się z LearnOpenGL i własnych dokumentów; tematy z wykładów porównujemy z tym, co już zrobione
Pytania o kod pisany z AI	Zasada zero tajemnic + dokumentacja + powtórka pytań kontrolnych w M9
Tutoriale używające GL 4.3+	Trzymać się 4.1; przy każdym nowym tutorialu sprawdzać wersję funkcji
Zbyt ambitny zakres	MUST najpierw (M0-M5 to już zaliczenie), efekty dokładane jako przełączalne moduły
Wydajność cieni i bloomu na Retinie	Post-process w połowie rozdzielczości, shadow mapa 1024-2048

14. Definicja sukcesu

Do końca listopada gra jest skończona. Na obronie: uruchamiam grę, przechodzę rundę w ok. 3 minuty, otwieram panel F1 i dla każdego z 15 tematów pokazuję przełącznik i fragment kodu, który go realizuje. Podczas code review potrafię wytłumaczyć każdy plik i odpowiadam na pytania swobodnie, bo dokumentacja w docs/ była moim materiałem do nauki. Kod jest czytelny, warstwowy i nadaje się do portfolio.